

JDBC

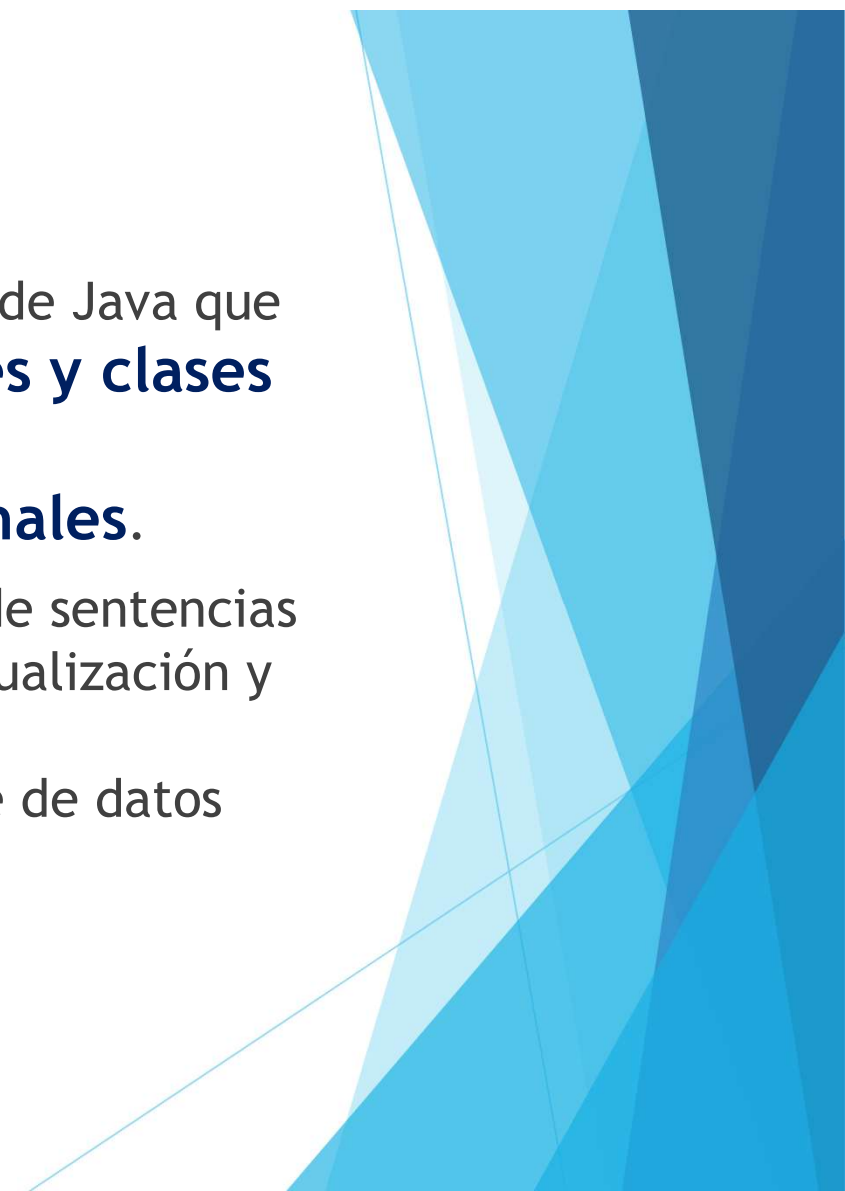
Java Database Connectivity



¿Qué es JDBC?

JDBC (Java Database Connectivity) es una API de Java que define un **conjunto estándar de interfaces y clases para permitir que las aplicaciones Java interactúen con bases de datos relacionales.**

Su principal propósito es facilitar la ejecución de sentencias SQL, permitiendo así la consulta, inserción, actualización y eliminación de datos de forma uniforme, independientemente del sistema gestor de base de datos utilizado.



¿Por qué utilizar JDBC?

Existen diferentes tecnologías para trabajar con bases de datos en aplicaciones Java.

Algunas alternativas utilizan frameworks ORM (Object-Relational Mapping), como Hibernate o JPA (Java Persistence API), que permiten mapear objetos Java directamente sobre tablas de la base de datos.

Sin embargo, JDBC continúa siendo la base del acceso a datos en Java y permite comprender de forma directa:

- ▶ cómo se establece una conexión
- ▶ cómo se ejecutan sentencias SQL
- ▶ cómo se administran transacciones
- ▶ y cómo se gestionan los recursos de acceso a datos.

Componentes de JDBC

- ▶ **Driver Manager:** Actúa como un gestor para diferentes controladores de bases de datos (drivers).
- ▶ **Driver:** Interfaz que cada proveedor de base de datos debe implementar para permitir que sus bases de datos se conecten mediante JDBC.
- ▶ **SQLException:** Clase que maneja errores y excepciones que pueden ocurrir durante la interacción con la base de datos.
- ▶ **Connection:** Representa una conexión activa con una base de datos y permite crear objetos Statement para ejecutar consultas SQL.

Componentes de JDBC (Cont.)

- ▶ **Statement:** Interface utilizada para ejecutar sentencias SQL estáticas. Hay tres tipos principales de Statements:
 - ▶ **Statement:** Para consultas SQL simples.
 - ▶ **PreparedStatement:** Para consultas SQL precompiladas con parámetros.
 - ▶ **CallableStatement:** Para ejecutar procedimientos almacenados.
- ▶ **ResultSet:** Representa el conjunto de resultados de una consulta SQL. Además, proporciona métodos para iterar y acceder a los datos devueltos.

¿Qué tipo de Statement usar en JDBC?

Ejecutar una sentencia SQL simple sin parámetros → **Utilizar Statement**

- ▶ Adecuado para sentencias fijas (sin datos externos o del usuario).
- ▶ No previene SQL Injection.
- ▶ Útil para scripts estáticos.

```
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery("select * from  
paises order by nom_pais");
```

¿Qué tipo de Statement usar en JDBC? (Cont.)

Ejecutar una sentencia SQL que incluya valores variables →

Utilizar PreparedStatement

- ▶ Permite definir parámetros con ?.
- ▶ Previene SQL Injection.
- ▶ Permite reutiliza con diferentes valores.
- ▶ Recomendado para operaciones tipo SELECT, INSERT, UPDATE o DELETE con filtros.

¿Qué tipo de Statement usar en JDBC? (Cont.)

```
PreparedStatement stmt =  
conn.prepareStatement("select * from provincias  
where cod_pais = ?");  
stmt.setString(1, "AR");  
ResultSet rs = stmt.executeQuery();
```

¿Qué tipo de Statement usar en JDBC? (Cont.)

Ejecutar un procedimiento almacenado → **Utilizar CallableStatement**

- ▶ Diseñado para invocar procedimientos definidos en la base de datos.
- ▶ Permite parámetros de entrada, salida y retorno.

¿Qué tipo de Statement usar en JDBC? (Cont.)

```
CallableStatement stmt = conn.prepareCall("{CALL  
ins_localidad(?,?,?,?)}");  
stmt.setString(1, "Córdoba");  
stmt.setString(2, "AR");  
stmt.setString(3, "X");  
stmt.registerOutParameter(4, java.sql.Types.INTEGER);  
stmt.executeUpdate();
```

¿Qué tipo de Statement usar en JDBC? (Cont.)

Requisito técnico	Tipo de Statement recomendado
Sentencia fija, sin parámetros	Statement
Sentencia con parámetros variables	PreparedStatement
Prevención de SQL Injection	PreparedStatement
Ejecución de procedimientos almacenados	CallableStatement

Métodos de ejecución en JDBC

Método	Utilización	Resultado
<code>executeQuery()</code>	Sentencias SELECT	Retorna un ResultSet
<code>executeUpdate()</code>	Sentencias INSERT, UPDATE, DELETE	Retorna la cantidad de filas afectadas
<code>execute()</code>	Procedimientos almacenados u operaciones generales	Retorna true o false según el resultado obtenido

NOTAS:

- ▶ En aplicaciones comunes, `executeQuery()` y `executeUpdate()` suelen ser los métodos más utilizados.
- ▶ El método `execute()` se utiliza generalmente en operaciones donde el tipo de resultado puede variar, como procedimientos almacenados.

Ciclo de vida de una conexión de JDBC

► Carga del Driver:

```
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
```

NOTA: Depende del controlador y la base de datos.

► Establecimiento de una conexión:

```
String url = "jdbc:sqlserver://localhost; databaseName=cdc;  
encrypt=false";  
String user = "sa";  
String pwd = "claveSa";
```

```
Connection connection = DriverManager.getConnection(url, user,  
pwd);
```

Ciclo de vida de una conexión de JDBC (Cont.)

- **Modo autocommit:** Determina si cada sentencia SQL se confirma automáticamente después de ejecutarse o si se requiere una confirmación explícita por parte del programador.

Característica	<code>connection.setAutoCommit(true)</code>	<code>connection.setAutoCommit(false)</code>
Modo de operación	Cada sentencia se confirma automáticamente.	Las sentencias deben confirmarse explícitamente.
Complejidad	Simple de usar.	Requiere gestión explícita de transacciones.
Control transaccional	Limitado a operaciones individuales.	Permite agrupar múltiples operaciones en una transacción.
Recuperación de errores	No se puede revertir una operación una vez ejecutada.	Se pueden revertir todas las operaciones de una transacción en caso de error.

Ciclo de vida de una conexión de JDBC (Cont.)

- ▶ **Creación de una sentencia SQL:**

```
Statement statement = connection.createStatement();
```

- ▶ **Ejecución de la sentencia:**

```
ResultSet resultSet = statement.executeQuery("SELECT *  
FROM personal");
```

- ▶ **Procesamiento del resultado:**

```
while (resultSet.next()) {  
    // Procesar cada fila  
}
```

- ▶ **Cierre de recursos:**

```
resultSet.close();  
statement.close();  
connection.close();
```

Ciclo de vida de una conexión de JDBC (Cont.)

¡¡Importante!!

- ▶ Abrir una conexión a una base de datos no implica únicamente crear un objeto Java.
- ▶ La aplicación debe establecer comunicación con el motor de base de datos, validar credenciales y reservar recursos.
- ▶ Por este motivo, las conexiones deben cerrarse correctamente y reutilizarse cuando sea posible.

Cierre automático de recursos en JDBC

Podemos usar **try-with-resources** para cerrar automáticamente los objetos que implementan la interfaz **AutoCloseable**, como son:

- ▶ Connection
- ▶ Statement / PreparedStatement / CallableStatement
- ▶ ResultSet

Estos recursos se cierran automáticamente al finalizar el bloque try, incluso si ocurre una excepción.

Cierre automático de recursos en JDBC (Cont.)

```
try (  
    Connection conn = DriverManager.getConnection(url, user, pwd);  
    Statement stmt = conn.createStatement();  
    ResultSet rs = stmt.executeQuery(query)  
) {  
    while (rs.next()) {  
        // Procesar cada fila  
    }  
}  
catch (SQLException e) {  
    // Tratamiento de la excepción  
}
```

Cierre automático de recursos en JDBC (Cont.)

```
try (Connection conn = DriverManager.getConnection(url, user, pwd)) {  
    conn.setAutoCommit(false); // Establezco manejo transaccional  
  
    try (Statement stmt = conn.createStatement()) {  
        stmt.executeUpdate(...);  
        conn.commit(); // Confirmar si todo fue exitoso  
    }  
    catch (SQLException ex) {  
        conn.rollback(); // Revertir si algo falla  
        // Tratamiento de la excepción  
    }  
}
```

Buenas prácticas en JDBC

- ▶ Usar **try-with-resources** siempre que sea posible para cerrar conexiones, sentencias y resultados automáticamente.
- ▶ Validar entradas del usuario y usar PreparedStatement o CallableStatement para evitar ataques de SQL Injection.
- ▶ Cerrar los recursos en el orden inverso en que se abrieron, si no se usa try-with-resources.
- ▶ Centralizar el manejo de errores y transacciones para tener un código más limpio y confiable.
- ▶ Registrar errores y eventos con una herramienta de logging (como java.util.logging, Log4j o SLF4J).
- ▶ Separar la lógica de acceso a datos del resto del código de la aplicación.

